Architettura della Pipeline

Generazione Procedurale di Terreni 3D e Dataset per l'Intelligenza Artificiale

Documentazione Tecnica - Versione 1.0

5 settembre 2026

1 Introduzione

Il presente documento descrive l'architettura tecnica e le scelte implementative di una pipeline sviluppata in **Blender 4.2** basata su *Geometry Nodes*. L'obiettivo del progetto è la generazione procedurale, automatizzata e scalabile di terreni geologici - definiti da crateri, montagne e coni alluvionali. L'output del sistema non è un semplice modello 3D, bensì un *Dataset* strutturato (composto da *Heightmaps* in formato EXR a 16/32-bit e *Splatmaps* RGB (quest'ultimo da confermare)) idoneo sia per l'utilizzo all'interno di Game Engine (es. Unreal Engine), sia per l'addestramento di modelli di Intelligenza Artificiale Generativa (es. Stable Diffusion XL).

2 Scelta Architetturale: Il Dominio 2.5D

Per garantire scalabilità ed efficienza computazionale, il sistema adotta un approccio **2.5D**. La generazione non avviene deformando liberamente un volume tridimensionale, ma si basa sul calcolo di *campi scalari* sul piano bidimensionale (XY). Il valore calcolato per ogni vertice viene assegnato in un'unica operazione finale all'asse Z della geometria tramite il nodo `Set Position`.

Questa modularità permette di trattare ogni elemento geologico (Montagne, Crateri, Coni alluvionali) come un modulo matematico isolato (Node Group), la cui altezza viene sommata o mascherata solo al termine del processo.

3 Libreria delle Feature Procedurali

Ogni conformazione geologica è stata ingegnerizzata tramite matematica vettoriale pura, eliminando la necessità di scultura manuale.

3.1 Montagne (Distanza da Curve e Domain Warping)

Le catene montuose sono generate calcolando la distanza (`Geometry Proximity`) dei vertici da una *Skeleton Line* (cresta montuosa). Per rendere il sistema 100% procedurale, la cresta non è disegnata a mano, ma è una `Curve Line` a cui viene applicato del rumore (*Noise Texture*). La pendenza dei versanti è controllata dinamicamente tramite una funzione di decadimento a potenza (`Power`), che genera un profilo a campana concava.

3.2 Crateri Vulcanici (Campi Radiali)

I crateri sono distribuiti proceduralmente tramite `Distribute Points on Faces` (algoritmo Poisson Disk). La morfologia del cratere è definita da una distanza radiale normalizzata tra 0.0 e 1.0, passata all'interno di una `Float Curve` che definisce in modo asimmetrico lo scavo del bacino (valori negativi) e l'innalzamento del bordo esterno (valori positivi).

3.3 Coni Alluvionali (Alluvial Fans)

Questa feature rappresenta la componente matematicamente più complessa del sistema. La morfologia e il posizionamento del deposito detritico direzionale sono stati ingegnerizzati combinando un motore vettoriale procedurale con un sistema di spawning autonomo (*Agent-Based*) per garantire l'accuratezza geologica.

Morfologia del Deposito (Motore Vettoriale)

Per simulare la geometria del deposito, è stato implementato un motore vettoriale che opera secondo i seguenti passaggi:

1. **Calcolo della direzione:** Estrazione del vettore direzione tra l'Apice (A) e la posizione corrente (P).
2. **Distanza frontale:** Utilizzo del **Prodotto Scalare** (*Dot Product*) tra il vettore normalizzato e la direzione del cono per ottenere la distanza frontale (r).
3. **Distanza laterale e limiti:** Calcolo della distanza laterale tramite sottrazione vettoriale e limitazione angolare mediante la funzione *Tangente*.
4. **Aspetto organico:** Introduzione di una distorsione del dominio (*Domain Warping*) basata su rumore (*Noise*) applicata alle maschere booleane di taglio, evitando bordi geometrici rettilinei e irrealistici.

Posizionamento Autonomo (Gradient-Based)

Per garantire che i coni si sviluppino in punti geologicamente coerenti, il sistema di spawning individua autonomamente le aree di deposizione ai piedi dei rilievi:

1. **Estrazione del Gradiente:** Il sistema calcola il vettore ortogonale di discesa sottraendo la posizione corrente dal punto più vicino sulla cresta montuosa (*Geometry Proximity*) e normalizzando il risultato.
2. **Fascia di Deposizione:** Tramite operazioni booleane sulle distanze, viene isolata una fascia perimetrale (un anello) alla base del rilievo montuoso.
3. **Istanziazione Orientata:** I punti sorgente generati in questa fascia (*Distribute Points on Faces*) ereditano il vettore di discesa tramite *Store Named Attribute*, garantendo che ogni cono alluvionale punti rigorosamente verso la valle.

3.4 Erosione Procedurale Morfologica

La simulazione fisicamente accurata dell'erosione idraulica richiede algoritmi iterativi (es. simulazione particellare delle precipitazioni) che risultano computazionalmente troppo onerosi per un flusso di lavoro procedurale in tempo reale. Pertanto, l'architettura adotta un approccio di *erosione morfologica* basata su campi di rumore.

L'algoritmo sviluppato opera per sottrazione (scavo) ed è così strutturato:

1. **Generazione del Pattern (Canalizzazione):** Viene utilizzato un campo *Noise Texture* ad alto dettaglio, mappato attraverso un `ColorRamp` con interpolazione ad alto contrasto. Questo restringe i gradienti del rumore trasformandoli in linee sottili, simulando visivamente i calanchi e i canali di scolo dell'acqua.
2. **Mascheramento (Context-Awareness):** Per evitare che l'erosione scavi le pianure, il pattern viene moltiplicato per una maschera di influenza, generata dall'unione logica (nodo `Maximum`) della *Mountain Mask* e dei bordi dei crateri.
3. **Sottrazione Altmetrica:** Il risultato viene moltiplicato per il parametro `Profondita Erosione` (gestibile da JSON) e sottratto dall'elevazione Z complessiva del terreno prima del calcolo finale della *Heightmap*.

In questo modo, i versanti montuosi presentano spaccature verticali naturali che seguono l'orografia del terreno, aumentando drasticamente la fedeltà visiva del Modello Digitale di Elevazione (DEM).

4 Blending Geologico e Maschere (Segmentazione)

La sovrapposizione casuale di feature procedurali può generare artefatti visivi (es. montagne che svettano dall'interno di un cratere). Per risolvere questo problema, il sistema implementa una logica *Context-Aware*.

4.1 Store Named Attribute

Ogni Node Group genera una propria maschera binaria (Bianco=1, Nero=0) che delimita la propria area di influenza. Questi dati vengono scritti sul flusso geometrico principale tramite `Store Named Attribute`.

4.2 L'Equazione di Blending

L'ordine di somma delle altezze rispetta le leggi geologiche:

- **Terreno Base + Montagne:** Le montagne, essendo orogenesi, spingono dal basso (Nodo: `Add`).
- **+ Crateri:** Gli impatti deformano e scavano la roccia preesistente (Nodo: `Add`).
- **+ Coni Alluvionali:** Essendo sedimenti, si adagiano per gravità colmando gli avvallamenti (Nodo: `Maximum`).

Inoltre, la cresta delle montagne legge la mappa combinata dei Crateri e dei Fan e azzera dinamicamente la propria altezza e pendenza in loro prossimità, garantendo un'integrazione fluida tramite il nodo `Blur Attribute`.

5 Pipeline di Esportazione e Automazione (Python)

Il sistema è progettato per essere interfacciato tramite uno script Python, trasformando Blender in un generatore batch headless.

5.1 Struttura Dati (JSON)

Tutti i parametri nodali (Seed, Altezze, Scale) sono mappati esternamente in un file JSON. Un algoritmo Python è stato sviluppato per associare dinamicamente i nomi in chiaro ai socket interni di Blender (es. `Socket_15`), rendendo il codice resiliente agli aggiornamenti del Node Tree.

5.2 Normalizzazione Assoluta per Intelligenza Artificiale

Per l'addestramento efficace di modelli di Intelligenza Artificiale generativa (quali SDXL), la coerenza matematica della scala dei valori di altezza costituisce un requisito fondamentale. Per evitare discrepanze durante l'esportazione, il processo di normalizzazione è stato centralizzato direttamente all'interno del *Geometry Node Tree*.

Tramite un nodo *Map Range*, l'intervallo delle altitudini è stato vincolato a limiti globali assoluti, definiti da un intervallo chiuso $[h_{\min}, h_{\max}]$. Questa operazione esegue

una normalizzazione lineare a monte, mappando biunivocamente i dati di elevazione nello spazio scalare standardizzato $[0.0, 1.0]$. Il dato normalizzato viene poi letto passivamente dallo Shader per l'esportazione finale della *heightmap*, garantendo che la scala di grigi mantenga una proporzione matematica assoluta in tutto il Dataset.

5.3 Metodi di Esportazione (Rendering Ortografico)

Lo script Python gestisce un nodo `Switch` nei Geometry Nodes per alternare dinamicamente i materiali in fase di render, bypassando il lento processo di Raycasting in favore del rendering GPU:

1. **Heightmap:** Esportata in formato EXR a 16-bit (scala di grigi). Formato ottimizzato per evitare il fenomeno del *terracing* e compatibile nativamente con i Dataloader (libreria PIL) per il Machine Learning.

2. **(Implementato ma da verificare se Utile) Splatmap:** Esportata in formato PNG 8-bit (RGB). Utilizza i canali colore per codificare le maschere esportate dai nodi (R=Crateri, G=Fan, B=Montagne), permettendo ai Game Engine di assegnare automaticamente PBR materials complessi.

6 Sviluppi Futuri

Le successive iterazioni della pipeline si concentreranno sull'integrazione del Dataset generato in ambienti *Real-Time*. In particolare:

- L'integrazione di Splatmap RGB all'interno di Unreal Engine 5 consente il pilotaggio automatico e parametrico del material blending. Sfruttando i singoli canali cromatici come maschere di distribuzione geologica, questa tecnica ottimizza l'assegnazione delle texture superficiali sul Landscape. Inoltre, i dati memorizzati nella Splatmap possono essere campionati direttamente dal PCG (Procedural Content Generation) Framework, automatizzando lo spawning e il posizionamento coerente di asset procedurali (quali fogliame, detriti e masse rocciose) in base alle caratteristiche morfologiche del terreno.

- L'implementazione di maschere supplementari basate sulla curvatura geometrica del terreno (*Curvature Maps*) permette di ottimizzare l'algoritmo di erosione idraulica. Attraverso l'analisi dei vettori di concavità e convessità della superficie, il sistema è in grado di simulare accuratamente la dinamica dei fluidi, veicolando l'accumulo procedurale dei sedimenti all'interno delle fessure e degli avvallamenti direttamente a livello di *shader*. **Tuttavia, l'effettiva integrazione di tali maschere richiederà una valutazione preliminare volta a verificarne la reale necessità, bilanciando il guadagno in termini di dettaglio visivo con l'incremento del costo computazionale in fase di calcolo.**

Architettura della Pipeline

Generazione Sintetica di Dataset Visivi tramite Stable Diffusion XL

Analisi Tecnica dei Componenti e del Codice di Inferenza

7 Generazione Sintetica di Paesaggi tramite Modelli Generativi

Nel contesto della generazione procedurale di terreni 3D e della creazione di dataset per l'Intelligenza Artificiale, l'utilizzo di modelli di *Deep Learning* text-to-image rappresenta un pilastro fondamentale per il superamento della scarsità di dati reali (*data scarcity*). Nello specifico, questa sezione analizza l'implementazione di uno script di inferenza basato sull'architettura **Stable Diffusion XL (SDXL)**, mirato alla sintesi di immagini di paesaggi ad alta risoluzione utilizzabili come riferimento geometrico o texture.

7.1 Il Modello Stable Diffusion XL (SDXL)

Stable Diffusion XL, sviluppato da Stability AI, è un modello generativo latente di diffusione (*Latent Diffusion Model - LDM*) progettato per la sintesi di immagini a partire da descrizioni testuali. Rispetto alle versioni precedenti (come SD 1.5 e 2.1), SDXL introduce un'architettura notevolmente potenziata, caratterizzata da un numero di parametri tre volte superiore (circa 3.5 miliardi per il modello base) e da un sistema di condizionamento testuale multiplo.

Il modello ha lo scopo di generare immagini fotorealistiche con una risoluzione nativa di 1024×1024 pixel, minimizzando gli artefatti geometrici e migliorando la fedeltà semantica rispetto al prompt testuale inserito dall'utente.

7.2 Componenti Chiave dell'Architettura della Pipeline

L'istanza `StableDiffusionXLPipeline` incapsula al suo interno diversi sotto-modelli indipendenti che cooperano durante il processo di inferenza:

- **Text Encoders (CLIP e OpenCLIP):** SDXL utilizza in parallelo due codificatori di testo (*CLIP ViT-L* e *OpenCLIP ViT-bigG*). Essi convertono i prompt testuali (positivi e negativi) in embedding vettoriali densi, catturando sia i dettagli testuali microscopici sia il contesto semantico macroscopico.
- **La UNet (Denoising Backbone):** È il nucleo computazionale del modello. Riceve una matrice di rumore casuale nello spazio latente e, condizionata dagli embedding del testo, predice iterativamente il rumore da rimuovere ad ogni *inference step*.
- **Il VAE (Variational Autoencoder):** L'immagine non viene generata direttamente nello spazio dei pixel (che richiederebbe troppa memoria), ma in uno spazio latente compresso di dimensioni $128 \times 128 \times 4$. Al termine del processo di denoising, il Decoder del VAE mappa questa rappresentazione latente nello spazio dei pixel effettivo, restituendo l'immagine finale a 1024×1024.

7.3 Analisi Tecnica del Codice di Inferenza

Di seguito viene analizzato l'algoritmo Python implementato per l'inizializzazione della pipeline e la generazione del paesaggio fantasy.

7.3.1 Configurazione dell'Ambiente e Gestione dei Path

```
import torch
from diffusers import StableDiffusionXLPipeline
import sys, os

sys.path.append(os.path.dirname(os.path.dirname(__file__)))

from utils.utils_method import get_full_path
from utils.constants import PATH_FOLDER_SDXL
```

Listing 1: Inizializzazione dei moduli e gestione del Python Path.

L'istruzione `sys.path.append` calcola dinamicamente la cartella radice (*root*) del progetto risalendo di due livelli rispetto alla posizione dello script corrente. Questa operazione previene l'errore `ModuleNotFoundError`, registrando nel sistema i moduli custom contenuti nella directory `utils`.

7.3.2 Ottimizzazione delle Risorse Hardware e Caricamento

```
model_id = "stabilityai/stable-diffusion-xl-base-1.0"

pipe = StableDiffusionXLPipeline.from_pretrained(
    model_id,
    torch_dtype=torch.float16,
    variant="fp16",
    use_safetensors=True
)
pipe = pipe.to("cuda")
```

Listing 2: Caricamento del modello SDXL in mezza precisione.

Al fine di consentire l'esecuzione dell'algoritmo su un hardware consumer (configurazione *Laptop*), sono state implementate tre strategie di ottimizzazione fondamentali:

1. `torch_dtype=torch.float16`: Converte i pesi del modello dal formato standard a 32-bit (FP32) alla mezza precisione (FP16), dimezzando l'allocazione della memoria VRAM richiesta sulla GPU senza compromettere la qualità visiva.

2. `variant="fp16"`: Forza il download esclusivo dei checkpoint pre-ottimizzati in virgola mobile a 16 bit, riducendo l'impatto sul network (circa 7GB totali anziché oltre 12GB).

3. `use_safetensors=True`: Utilizza il formato di archiviazione *Safetensors* in sostituzione dei vecchi file Pickle, garantendo una velocità di caricamento superiore ed escludendo vulnerabilità legate all'esecuzione di codice arbitrario.

4. `pipe.to("cuda")`: Sposta i tensori della pipeline dall'host (CPU) al device (GPU NVIDIA) per sfruttare i core di calcolo parallelo Tensor Core durante l'inferenza.

7.3.3 Parametrizzazione del Processo di Diffusione e Output

```
prompt = "A breathtaking procedural fantasy landscape, towering majestic
    mountains, lush green valley, volumetric lighting, photorealistic, 8k
    resolution,
highly detailed"
negative_prompt = "blurry, low quality, distorted, bad composition,
    watermark"

image = pipe(
    prompt=prompt,
    negative_prompt=negative_prompt,
    num_inference_steps=40,
    guidance_scale=7.5
).images[0]

output_path = get_full_path(PATH_FOLDER_SDXL, "paesaggio_generato.png")
image.save(output_path)
```

Listing 3: Generazione sintetica dell'immagine e salvataggio.

Il processo di generazione viene regolato tramite i seguenti iperparametri di inferenza:

- `num_inference_steps=40`: Rappresenta il numero di iterazioni di rimozione del rumore eseguite dalla UNet. Un valore pari a 40 assicura un bilanciamento ottimale tra convergenza della qualità d'immagine e tempo di calcolo su GPU mobile.
- `guidance_scale=7.5`: Definisce il coefficiente della *Classifier-Free Guidance (CFG)*. Regola la forza con cui il modello aderisce alla descrizione testuale del prompt rispetto alla generazione libera. Un valore di 7.5 è lo standard industriale per evitare saturazioni cromatiche mantenendo un'elevata fedeltà semantica.

Infine, la matrice latente decodificata viene estratta come oggetto immagine PIL salvata sul disco locale tramite la funzione `get_full_path`, mappando dinamicamente il file all'interno della directory di destinazione `PATH_FOLDER_SDXL`.

7.4 Anatomia dell'Oggetto Pipeline: Deconnessione della Scatola Nera

Nella libreria `diffusers`, l'oggetto `StableDiffusionXLPipeline` (indicato nel codice come `pipe`) non deve essere interpretato come un blocco funzionale monolitico o una *scatola nera*. Al contrario, esso funge da orchestratore software di alto livello che incapsula e coordina quattro componenti neurali e matematici distinti. Ciascuno di questi moduli ricopre un ruolo specifico e interdipendente durante il processo iterativo di generazione dell'immagine:

1. **Doppio Text Encoder (CLIP ViT-L e OpenCLIP ViT-bigG):** A differenza delle architetture di generazione precedenti che si affidavano a un singolo codificatore, SDXL impiega in parallelo due distinti modelli di linguaggio per trasformare il prompt testuale di input in vettori matematici densi (*embeddings*).
    - *CLIP ViT-L* (sviluppato da OpenAI) si focalizza sulla cattura delle relazioni semantiche standard e dei concetti base del testo.
    - *OpenCLIP ViT-bigG* (modello open-source di dimensioni maggiori) estrae dettagli testuali microscopici e sfumature stilistiche complesse.

I vettori generati da entrambi i modelli vengono concatenati e fusi insieme. Questo permette di mappare la descrizione del paesaggio fantasy in uno spazio vettoriale condiviso ad altissima fedeltà, che guiderà i successivi stadi visivi.

2. **Scheduler (Il Campionatore):** Lo *Scheduler* rappresenta il motore matematico della pipeline. Non è una rete neurale, bensì un insieme di algoritmi deterministici e stocastici (come *Euler Discrete*, *DDIM*, o *DPM++*) deputati a regolare la traiettoria di aggiunta e rimozione del rumore gaussiano (*noise*). Durante la fase di inferenza, lo Scheduler definisce la discretizzazione temporale e la dimensione esatta dei passi matematici necessari per guidare la transizione controllata da una matrice iniziale di puro rumore casuale (analogo al rumore televisivo) a una rappresentazione visiva strutturata e coerente.

3. **UNet (Denoising Backbone):** Rappresenta il nucleo computazionale e il vero "cervello" visivo del modello. Al fine di ottimizzare l'efficienza hardware ed evitare la saturazione della memoria (critica in architetture consumer come i laptop), la UNet non opera direttamente sulla risoluzione nativa in pixel dell'immagine. Al contrario, essa lavora all'interno di uno **spazio latente compresso**, riducendo la dimensionalità dei dati a una frazione dell'originale ($128 \times 128 \times 4$). Ad ogni iterazione (corrispondente ai 40 passi computazionali impostati nel codice), la UNet analizza la matrice latente corrente e, condizionata dagli embedding del testo tramite meccanismi di *cross-attention*, effettua una stima computazionale del rumore residuo presente nella matrice stessa, fornendo lo spettro da sottrarre per lo step successivo.

4. **VAE (Variational Autoencoder):** Al termine dei 40 cicli di denoising operati dalla UNet e dallo Scheduler, il processo latente giunge a convergenza. Tuttavia, l'output ottenuto è ancora confinato all'interno dello spazio latente: si tratta di una matrice multidimensionale astratta, priva di significato biologico o visivo per l'occhio umano. A questo punto interviene il componente di decodifica del *VAE*. Il modulo riceve la matrice latente pulita e ne effettua l'up-sampling spaziale e semantico, traducendo i costrutti matematici astratti nello spazio dei pixel effettivo. Il risultato finale di questa operazione è l'immagine bidimensionale standard (RGB) visualizzabile e memorizzabile sul disco.

Compreso il funzionamento interno dell'oggetto `pipe`, risulta evidente come i parametri passati alla pipeline (quali la precisione numerica `torch.float16` o il numero di passi di campionamento) vadano a impattare direttamente sui singoli moduli interni qui descritti, modificandone il comportamento e l'occupazione di memoria in fase di calcolo.

7.5 Meccanismi Avanzati e Micro-Condizionamenti in SDXL

Per comprendere appieno il comportamento computazionale della pipeline, è necessario evidenziare due innovazioni teoriche introdotte dall'architettura SDXL rispetto ai modelli di generazione latente di precedente generazione:

- **Meccanismo di Cross-Attention:** Il collegamento tra il doppio Text Encoder e la UNet avviene attraverso strati di *cross-attention*. Matematicamente, gli embedding del testo vengono trasformati in vettori di *Key* (K) e *Value* (V), mentre le caratteristiche spaziali della matrice latente estratte dalla UNet diventano vettori di *Query* (Q). Il calcolo dell'attenzione, espresso dalla formula standard

$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$, permette al modello di mappare con precisione millimetrica i token testuali (ad esempio, l'entità "bottle") sulle coordinate spaziali corrette della matrice latente, guidando la genesi delle forme.

- **Micro-condizionamento geometrico:** Uno dei problemi storici dei modelli di diffusione era la tendenza a generare soggetti tagliati o con artefatti dovuti al ridimensionamento delle immagini di *training*. SDXL risolve questo problema alimentando la UNet con metadati geometrici aggiuntivi durante l'inferenza, quali la risoluzione originale dell'immagine (*Original Size*) e le coordinate di ritaglio (*Crop Coordinates*). Questo condizionamento supplementare forza il modello a campionare i pixel simulando un'inquadratura fotografica nativa e centrata, incrementando drasticamente il fotorealismo dell'output.

7.6 Studio Approfondito dei Componenti Neurali della Pipeline

L'efficacia e la stabilità computazionale di `StableDiffusionXLPipeline` risiedono nell'interazione dei suoi quattro moduli costitutivi. Di seguito viene presentata un'analisi dettagliata del funzionamento teorico e strutturale di ciascun componente.

7.6.1 Il Sistema di Tokenizzazione e Doppia Codifica del Testo (Text Encoders)

Il testo immesso dall'utente sotto forma di stringa di caratteri non può essere elaborato direttamente da una rete neurale. Il processo di conversione avviene in due fasi distinte:

1. **Tokenizzazione:** Il *Tokenizer* seziona la stringa in unità semantiche minime chiamate *token* (parole o sotto-parole) e associa a ciascuna di esse un indice numerico univoco basato su un vocabolario predefinito.
2. **Codifica (Embedding):** Gli indici numerici vengono mappati all'interno di uno spazio vettoriale continuo attraverso due modelli di linguaggio operanti in parallelo:
    - **CLIP ViT-L/14 (OpenAI):** Genera una rappresentazione vettoriale con una dimensione di embedding di 768 canali. Si focalizza sulle relazioni semantiche di base e sulla struttura sintattica standard.
    - **OpenCLIP ViT-bigG/14 (LAION):** Caratterizzato da un'architettura notevolmente più capiente, proietta il testo in una dimensione di embedding di 1280 canali. Questo modulo cattura dettagli visivi microscopici, attributi stilistici e sfumature ambientali complesse.

I vettori di output estratti dai layer intermedi di entrambi i modelli vengono concatenati e fusi insieme, generando un tensore di condizionamento testuale unificato. Questo approccio a doppia architettura conferisce a SDXL una comprensione del prompt straordinariamente superiore rispetto ai modelli del passato, traducendo concetti complessi in coordinate matematiche di massima precisione.

7.6.2 Lo Scheduler (Il Campionatore Deterministico/Stocastico)

Lo *Scheduler* agisce come il motore matematico e il regolatore termodinamico della pipeline. A differenza degli altri tre componenti, non è una rete neurale basata sull'apprendimento di pesi sinaptici, ma un insieme di equazioni differenziali e algoritmi di discretizzazione (es. *Euler Discrete, DPM++ o DDIM*).

Il suo scopo principale è governare la traiettoria di inversione del tempo nel processo di diffusione. Durante l'addestramento, il modello impara come il rumore gaussiano distrugge progressivamente un'immagine. In fase di inferenza, lo Scheduler calcola la frazione esatta di rumore teorico che deve essere rimossa ad ogni singolo passo (i 40 step definiti nello script), aggiornando la varianza e la media del tensore latente. Esso funge da guida matematica, assicurando che la UNet non compia deviazioni caotiche durante la ricostruzione del soggetto dal caos iniziale del "rumore televisivo".

7.6.3 La UNet (Il Nucleo di Denoising nello Spazio Latente)

La *UNet* rappresenta la spina dorsale computazionale e il nucleo visivo del modello generativo, deputata alla predizione del rumore. Dal punto di vista strutturale, è una rete neurale convoluzionale simmetrica caratterizzata da blocchi di campionamento verso il basso (*Downsampling*), blocchi di campionamento verso l'alto (*Upsampling*) e collegamenti diretti (*Skip Connections*) che preservano le informazioni spaziali ad alta frequenza.

La caratteristica fondamentale della UNet di SDXL risiede nell'ottimizzazione delle risorse hardware: essa opera esclusivamente all'interno di un **spazio latente bidimensionale compresso** gestito a 4 canali ($128 \times 128 \times 4$). Ad ogni iterazione, la UNet riceve il tensore latente rumoroso e, tramite meccanismi di *Cross-Attention*, confronta la disposizione spaziale temporanea dei pixel latenti con i vettori di embedding generati dai Text Encoders. Sulla base di questo confronto, la UNet stima matematicamente quale sia il rumore di disturbo accumulato nella matrice per poterlo sottrarre, permettendo l'estrazione progressiva di forme geometriche coerenti (i contorni della bottiglia, la texture del tavolo).

7.6.4 Il VAE (Variational Autoencoder)

Il *Variational Autoencoder* è un modello composto da due sezioni neurali speculari (un *Encoder* e un *Decoder*) il cui fine è la compressione e la successiva decompressione dell'informazione visiva. Durante la fase di generazione dell'immagine, viene impiegato esclusivamente il modulo **Decoder**.

Al termine dei 40 passaggi iterativi regolati da UNet e Scheduler, la matrice latente ha raggiunto lo stato di convergenza desiderato, risultando completamente priva di rumore di fondo. Tuttavia, questo output risiede ancora in uno spazio matematico astratto non visualizzabile. Il Decoder del VAE assume il ruolo di traduttore e sintetizzatore finale: prende il tensore latente compresso di dimensioni $128 \times 128 \times 4$ ed esegue un'operazione di espansione spaziale (*Up-sampling*) di un fattore pari a 8. Questa decompressione mappa i vettori latenti nello spazio effettivo dei pixel reali, restituendo un'immagine standard RGB a tre canali alla risoluzione nativa di 1024×1024 pixel.